

From vOptGeneric.jl to MultiObjectiveAlgorithms.jl

JuliaDays 2023 | 4-6 October 2023. Paris, France.

Oscar DOWSON (Auckland, New Zealand)

Xavier GANDIBLEUX (Nantes, France)

Gokhan KOF (Istanbul, Turkey)

From vOptGeneric.jl . . .

- ANR-DFG Research Project (2015-2019)



- Optimization problems targeted:

Multiobjective linear Optimization Problems (MOP)

Multiobjective linear Optimization Problems targeted

Computing Y_N for Multiobjective linear Optimization Problems (MOP):

$$\begin{array}{ll} \min F(x) & = \quad Cx \\ \text{s/t} \quad Tx & \leqslant d \end{array}$$

$$x \in \mathbb{R}^{n_1}$$

Multi Objective
Linear
Problem (**MOLP**)

$$x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2}$$

Multi Objective
Mixed-Integer Linear
Problem (**MOMILP**)

$$x \in \mathbb{Z}^{n_2}$$

Multi Objective
Integer
Problem (**MOIP**)

MO(M)ILP + structure

MultiObjective Combinatorial Optimization (**MOCO**)

where:

$T \in \mathbb{Z}^{m \times n} \longrightarrow m \text{ constraints, } i = 1, \dots, m$

$C \in \mathbb{Z}^{n \times p} \longrightarrow \text{the objective matrix}$

$X = \{x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2} \mid Tx \leqslant d\} \subseteq \mathbb{R}^n \longrightarrow \text{the set of feasible solutions}$

$Y = F(X) \subseteq \mathbb{R}^p \longrightarrow \text{the set of images}$

Sets Y_N and X_E

Computing Y_N for Multiobjective linear Optimization Problems (MOP):

$$\begin{array}{ll}\min & F(x) = Cx \\ \text{s/t} & Tx \leq d\end{array}$$

$$x \in \mathbb{R}^{n_1}$$

$$x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2}$$

$$x \in \mathbb{Z}^{n_2}$$

$$y = F(x)$$

$y^* \in Y$ is **nondominated**, if $\nexists y \in Y$ such that $y_i \leq y_i^*, \forall i$ and $y \neq y^*$.

Y_N is the set of nondominated points.

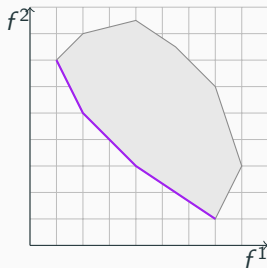
$x^* \in X$ is **efficient** if y^* is nondominated.

X_E is a complete set of efficient solutions.

Examples of Y and Y_N when $p = 2$

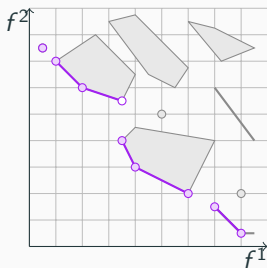
Computing Y_N for Multiobjective (linear) Optimization Problems (MOP):

$$x \in \mathbb{R}^{n_1}$$



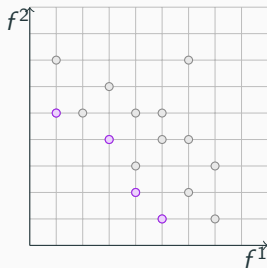
Continuous
nondominated set
(edges)

$$x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2}$$



Nondominated set
composed of edges that
are either closed,
half-open, open or
reduced to a point

$$x \in \mathbb{Z}^{n_2}$$



Discrete set of
nondominated
points

Purposes of vOptSolver

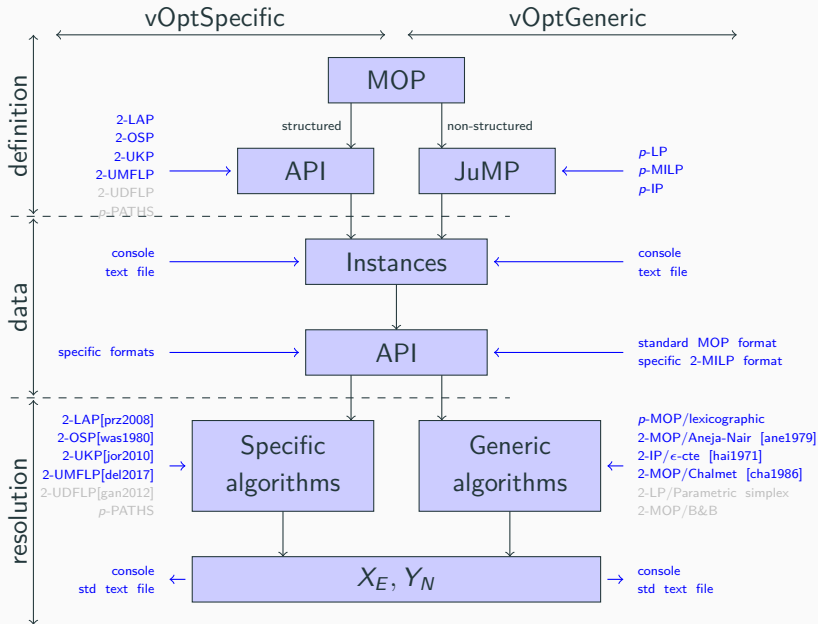
Natural and **intuitive** use for mathematicians, informaticians, engineers

- **Research needs:**
support and primitives for the development of new algorithms
- **Solving needs:**
methods and algorithms for performing numerical experiments
- **Pedagogic needs:**
environment for practicing of theories and algorithms

Easy to formulate a problem, provide the data, solve a problem, collect the outputs, analyze the solutions.

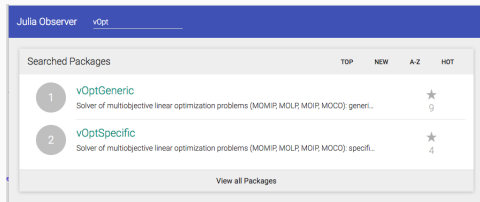
- Efficient, flexible, evolutive solver
- Free, open source, multi-platform, reusing existing specifications
- Easy installation, no need of being expert in computer science

Design of vOptSolver



Packages of vOptSolver

- Two julia packages:



- First release in July 2017

Xavier Gandibleux, Gauthier Soleilhac, Anthony Przybylski, Flavien Lucas, Stefan Ruzika, Pascal Halfmann. vOptSolver, a "get and run" solver of multiobjective linear optimization problems built on Julia and JuMP. MCDM2017: 24th International Conference on Multiple Criteria Decision Making. July 10-14, 2017. Ottawa (Canada).

Xavier Gandibleux, Gauthier Soleilhac, Anthony Przybylski, Stefan Ruzika. vOptSolver: an open source software environment for multiobjective mathematical optimization. IFORS2017: 21st Conference of the International Federation of Operational Research Societies. July 17-21, 2017. Quebec City (Canada).

Example: 2-LAP and vOptGeneric

$$\left[\begin{array}{ll} \min z^k & = \sum_{i=1}^n \sum_{j=1}^n c_{ij}^k x_{ij} \quad k = 1, \dots, 2 \\ s/c & \sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, n \\ & \sum_{j=1}^n x_{ij} = 1 \quad i = 1, \dots, n \\ & x_{ij} \in \{0, 1\} \quad i, j = 1, \dots, n \end{array} \right]$$

```
1  C1 = [ 3 9 0 0 6 ;
2         16 0 6 12 19 ;
3         2 7 11 15 8 ;
4         4 11 7 16 3 ;
5         2 5 1 9 0 ]
6
7  C2 = [16 5 6 19 12 ;
8         15 7 13 7 7 ;
9         1 2 13 2 3 ;
10        14 7 8 1 7 ;
11        10 10 1 0 0 ]
12
13 n = size(C2,1)
```

Example: 2-LAP and vOptGeneric

Algorithm: ϵ -constraint

Yacov V. Haimes, Leon S. Lasdon, David A. Wismer: On a bicriterion formation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man and Cybernetics*. Volume SMC-1, Issue 3, 296-297, 1971.

MILP: GLPK

Output: $X_E \subseteq \mathbb{N}^n$, $Y_N \subseteq \mathbb{Z}^2$

Program:

```
1  using JuMP, GLPK, vOptGeneric
2  m = vModel( GLPK.Optimizer )
3  @variable( m, x[1:n,1:n] , Bin )
4  @addobjective( m, Min, sum( C1[i,j]*x[i,j] for i=1:n,j=1:n ))
5  @addobjective( m, Min, sum( C2[i,j]*x[i,j] for i=1:n,j=1:n ))
6  @constraint( m, cols[i=1:n], sum(x[i,j] for j=1:n) == 1 )
7  @constraint( m, rows[j=1:n], sum(x[i,j] for i=1:n) == 1 )
8  vSolve( m, method = :epsilon , step = 1.0 )
9  printX_E( m )
10 getY_N( m )
```

Timeline

- 2015: Kick-off of vOpt project
- 2017: Introduction of vOptSolver (vOptGeneric, vOptSpecific)
- 2018: Julia 1.0
- 2019: End of vOpt project
- 2019: First discussions on discourse/new design of MOO in JuMP
- 2022: JuMP 1.0
- 2023: Introduction of MultiObjectiveAlgorithms
- 2023: End of vOptGeneric

...to MultiObjectiveAlgorithms.jl

The screenshot shows a web browser window displaying a GitHub issue page. The browser's address bar shows the URL `github.com/jump-dev/JuMP.jl/issues/2099`. The GitHub header includes the repository name `jump-dev / JuMP.jl` and a search bar. Below the header, navigation tabs for `Code`, `Issues` (selected), `Pull requests`, `Discussions`, `Actions`, `Security`, `Insights`, and `Settings` are visible. The issue title is **Multiobjective support in JuMP #2099**, with a status of **Closed**. A comment by user `odow` is shown, dated Nov 9, 2019. The comment text reads: "This issue is for a discussion on introducing multiobjective support to JuMP (and MOI)." Below this, the section **Passing multiple objectives to the solver** is highlighted. It states: "There are two possibilities:" and lists two options. Option 1 involves extending the `@objective` macro to support vector-valued functions. Option 2 involves adding an `index` keyword. Both options include code snippets for defining the objective function. On the right side of the issue, there are sections for **Assignees** (No one), **Labels** (Type: Feature request), **Milestone** (1.x), and **Development** (Successfully merging issue).

jump-dev / JuMP.jl

Search Type [/](#) to search

<> Code Issues 9 Pull requests 1 Discussions Actions Security Insights Settings

Multiobjective support in JuMP #2099

Closed odow opened this issue on Nov 9, 2019 · 36 comments · Fixed by #3176

odow commented on Nov 9, 2019 · edited Member

This issue is for a discussion on introducing multiobjective support to JuMP (and MOI).

Passing multiple objectives to the solver

There are two possibilities:

1. Extending `@objective` to support vector-valued functions, so the user would write:

```
@objective(model, Min, C * x)
```
2. Adding an `index` keyword, so the user would write:

```
@objective(model, Min, C[1, :] * x, index = 1)  
@objective(model, Min, C[2, :] * x, index = 2)
```

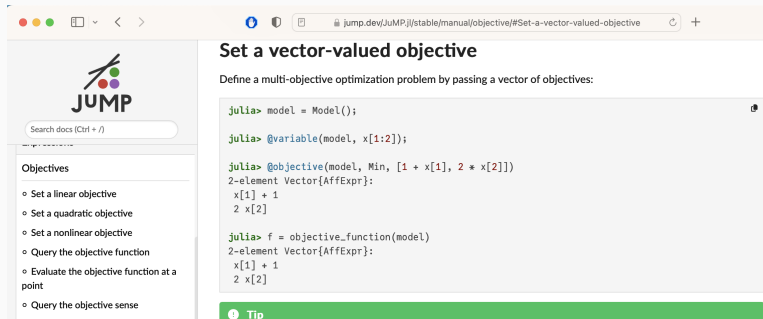
Assignees
No one—assign yourself

Labels
Type: Feature request

Milestone
1.x

Development
Successfully merging issue.
Add support for jump-dev/JuMP.jl

Vector-valued objectives



The screenshot shows a web browser displaying the JUMP.jl documentation page. The page title is "Set a vector-valued objective". Below the title, it says "Define a multi-objective optimization problem by passing a vector of objectives:". The main content area shows Julia code for setting up a multi-objective problem. On the left, there is a sidebar with the JUMP logo and a list of "Objectives" topics. At the bottom, there is a green "Tip" box.

Objectives

- Set a linear objective
- Set a quadratic objective
- Set a nonlinear objective
- Query the objective function
- Evaluate the objective function at a point
- Query the objective sense

Set a vector-valued objective

Define a multi-objective optimization problem by passing a vector of objectives:

```
julia> model = Model();

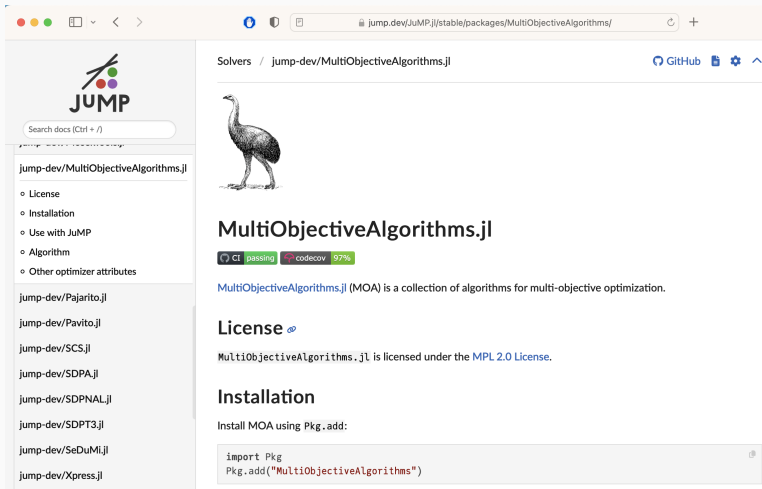
julia> @variable(model, x[1:2]);

julia> @objective(model, Min, [1 + x[1], 2 * x[2]])
2-element Vector{AffExpr}:
 x[1] + 1
 2 x[2]

julia> f = objective_function(model)
2-element Vector{AffExpr}:
 x[1] + 1
 2 x[2]
```

Tip

MultiObjectiveAlgorithms.jl



The screenshot shows a web browser displaying the documentation for `MultiObjectiveAlgorithms.jl`. The browser's address bar shows the URL `jump.dev/JuMP.jl/stable/packages/MultiObjectiveAlgorithms/`. The page features a sidebar on the left with the JUMP logo and a search bar. The main content area includes a navigation menu with links to `License`, `Installation`, `Use with JuMP`, `Algorithm`, and `Other optimizer attributes`. The `Installation` section is currently selected, showing the text "Install MOA using `Pkg.add`:" followed by a code block containing the Julia command `import Pkg; Pkg.add("MultiObjectiveAlgorithms")`. Above the title, there is a small illustration of an ostrich and a status bar indicating that the CI is passing and codecov coverage is 97%.

jump.dev/JuMP.jl/stable/packages/MultiObjectiveAlgorithms/

Solvers / jump-dev/MultiObjectiveAlgorithms.jl

GitHub

MultiObjectiveAlgorithms.jl

CI passing codecov 97%

MultiObjectiveAlgorithms.jl (MOA) is a collection of algorithms for multi-objective optimization.

License

MultiObjectiveAlgorithms.jl is licensed under the [MPL 2.0 License](#).

Installation

Install MOA using `Pkg.add`:

```
import Pkg
Pkg.add("MultiObjectiveAlgorithms")
```

Example: 2-LAP and vOptGeneric

```
1 using JuMP, HiGHS, vOptGeneric
2 model = vModel(HiGHS.Optimizer)
3 @variable(model, x[1:n,1:n], Bin)
4 @addobjective(model, Min, sum(C1 .* x))
5 @addobjective(model, Min, sum(C2 .* x))
6 @constraint(model, [i in 1:n], sum(x[i,:]) == 1)
7 @constraint(model, [j in 1:n], sum(x[:,j]) == 1)
8 vSolve(model; method = :epsilon, step = 1.0)
9 points = getY_N(model)
```

```
1 using JuMP, HiGHS
2 import MultiObjectiveAlgorithms as MOA
3 model = Model{() -> MOA.Optimizer}(HiGHS.Optimizer())
4 @variable(model, x[1:n, 1:n], Bin)
5 @objective(model, Min, [sum(C1 .* x), sum(C2 .* x)])
6 @constraint(model, [i in 1:n], sum(x[i,:]) == 1)
7 @constraint(model, [j in 1:n], sum(x[:,j]) == 1)
8 set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
9 set_attribute(model, MOA.EpsilonConstraintStep(), 1.0)
10 optimize!(model)
11 points = [objective_value(model; result = i) for
12           i in 1:result_count(model)]
```

Algorithms available

A number of algorithms are available

```
1 set_attribute(model, MOA.Algorithm(), ...)
```

MOA.Chalmet()

MOA.Dichotomy()

MOA.DominguezRios()

MOA.EpsilonConstraint()

MOA.Hierarchical()

MOA.KirlikSayin()

MOA.Lexicographic() [default]

MOA.TambyVanderpooten()

Optimizer attributes to setup

Algorithms can be configured via attributes

```
1 set_attribute(model, attribute, value)
```

```
MOA.EpsilonConstraintStep()  
MOA.LexicographicAllPermutations()  
MOA.ObjectiveAbsoluteTolerance(index::Int)  
MOA.ObjectivePriority(index::Int)  
MOA.ObjectiveRelativeTolerance(index::Int)  
MOA.ObjectiveWeight(index::Int)  
MOA.SolutionLimit()  
MOI.TimeLimitSec()
```

Adding a new algorithm to MOA

Adding a new algorithm is easy...

```
1 struct MyNewAlgorithm <: MOA.AbstractAlgorithm end
2
3 function MOA.optimize_multiobjective!(
4     ::MyNewAlgorithm,
5     model::Optimizer,
6 )
7     status = MOI.OTHER_ERROR
8     solutions = MOA.SolutionPoint[]
9     # ... your algorithm here ...
10    return status, solutions
11 end
```

Adding a new algorithm to MOA

...and requires little code and doesn't require deep knowledge of JuMP/MathOptInterface.

Filename	# Lines	Author
Chalmet.jl	117	Gokhan
Dichotomy.jl	127	
DominguezRios.jl	226	Gokhan
EpsilonConstraint.jl	131	
Hierarchical.jl	129	
KirlikSayin.jl	190	Gokhan
Lexicographic.jl	111	
TambyVanderpooten.jl	211	Gokhan

lines includes license header, docstrings, option handling, comments, blank lines...



Gökhan Kof
kofgokhan

Experiments

Tested on Julia v1.8.5 using HiGHS v1.5.3 on a machine with Intel(R) Xeon(R) CPU E5-2640 @ 2.50GHz

Instances with 3 objectives (<https://www.mcdmsociety.org/>)

CPU times in seconds. Average of 3 runs:

	3-01UKP (n=20)	3-LAP (n=10)	3-ILP (m=20, n=10)
Y_N	35	161	15
<i>Algorithms:</i>			
Kirlik & Sayin	6.7585	27.6896	6.7628
Dominguez-Rios	10.5700	62.8003	8.7380
Tamby & Vanderpooten	6.6883	21.6311	5.6501

Next steps

- From JuMP and MOA point of view:
 - everything is in place
- From a MOO perspective:
 - Algorithms
 - a bi-objective simplex based on HiGHS (available, testing)
 - a branch-and-xx ?
 - Documentation
 - a wider user documentation ?
 - a teaching material ?
 - Pending
 - algorithms in vOptSpecific?

Resources

- JuMP

<https://jump.dev/>

- MultiObjectiveAlgorithms (MOA)

<https://github.com/jump-dev/MultiObjectiveAlgorithms.jl>

- Examples

https://jump.dev/JuMP.jl/stable/tutorials/linear/multi_objective_knapsack/

https://jump.dev/JuMP.jl/stable/tutorials/linear/multi_objective_examples/

- vOptSolver (vOptGeneric and vOptSpecific)

<https://github.com/vOptSolver>

vOptGeneric.jl is a JuMP extension package for modeling and solving multi-objective linear optimization problems. It was created as part of the ANR/DFG research project vOpt (2015-2019). Inspired by the lessons learned in vOptGeneric, we added first-class support for multi-objective optimization problems to JuMP and MathOptInterface. In addition, we developed a meta-solver, MultiObjectiveAlgorithms.jl (MOA), which implements a number of algorithms for solving multi-objective optimization problems. The new features and MOA were released in February, 2023. In this talk, we discuss vOptGeneric, the transition to first-class support in JuMP, and our plans for the future.

by

Oscar Dowson (Auckland, New Zealand),
Xavier Gandibleux (Nantes, France),
Gokhan Kof (Istanbul, Turkey).